Software Design Document

- CrossNote

Datum:	23.01.2026
Version:	1.0
Prüfer:	Nicolas Kauer
Freigegeben von:	Jakob Scheuermeyer (Qualitätsmanager)

Changelog:

1.0: Deckblatt, grobe Ausarbeitung

Gerhard Uebe, Nicolas Kauer, Jakob Scheuermeyer, Kai Seitz

1. Zweck des Software Design Documents

Dieses Software-Design Document beschreibt die technische Ausgestaltung und Umsetzung der Anwendung CrossNote.
Es konkretisiert die zuvor definierte Softwarearchitektur sowie die funktionalen und nicht-funktionalen Anforderungen und dient als verbindliche Grundlage für Implementierung, Test und Wartung der Software

Das Dokument beantwortet insbesondere die Frage, wie die einzelnen Funktionen von CrossNote technisch realisiert werden.

2. Ziele des Software Design Documents

Das SDD verfolgt folgende Ziele:

- **Strukturierung**
  Detaillierte Beschreibung aller Softwarekomponenten, Module und deren Verantwortlichkeiten.
- **Kommunikation**
  Einheitliche technische Grundlage für Entwickler, Tester und Projektbeteiligte.
- **Nachvollziehbarkeit**
  Sicherstellung einer klaren Verbindung zwischen Anforderungen, Architektur und Implementierung
  .

3. Übersicht der Softwarearchitektur

3.1 Architektureller Überblick

CrossNote ist als Offline-First Notizanwendung konzipiert.
Die Architektur folgt dem Prinzip der klaren Trennung von Benutzeroberfläche, Anwendungslogik und Datenhaltung, um Wartbarkeit, Erweiterbarkeit und Plattformunabhängigkeit sicherzustellen.

Die Software gliedert sich in folgende Hauptbereiche:

- **UI-Schicht (plattformabhängig)**
  Realisiert die Benutzerinteraktion auf Desktop (Windows/Linux) und Android.
- **Core-Logik (plattformneutral, JVM-basiert)**
  Enthält die zentrale Geschäftslogik für Notizen, Tags, Suche, Revisionen und Papierkorb.
- **Persistenzschicht**
  Verantwortlich für lokale Speicherung, Versionierung und Wiederherstellung von Daten.
- **Synchronisationsmodul**
  Ermöglicht späteren Abgleich mit einem externen Dienst inklusive Konfliktbehandlung.

3.2 Architekturdiagramme

Zur Visualisierung der Architektur werden folgende UML-Diagramme eingesetzt:

- Paketdiagramm zur Darstellung der Modulstruktur
- Klassendiagramme für zentrale Daten- und Logik Klassen
- Sequenzdiagramme für zentrale Anwendungsfälle (z. B. Notiz speichern, Notiz wiederherstellen)

4. Modulübersicht

Die Anwendung CrossNote besteht aus den folgenden zentralen Modulen:

- Note Management Module
- Tag & Notebook Modul
- Search Modul
- Revision & History Modul
- Trash (Papierkorb) Modul
- Persistence Module
- Sync Module

Jedes Modul ist logisch abgeschlossen und kommuniziert über klar definierte Schnittstellen mit anderen Modulen.

5. Detaillierte Modulbeschreibungen

5.1 Note Management Module

Aufgaben:

- Erstellen, Bearbeiten und Löschen von Notizen
- Verwaltung von Metadaten (Titel, Zeitstempel, Status)

Schnittstellen:

- Übergabe von Notizobjekten an Persistenzmodul
- Zugriff durch UI-Schicht
- Zusammenarbeit mit Revision-Modul

Datenmodelle:

- Note
    - ID
    - Titel
    - Inhalt
    - Erstellungs- und Änderungsdatum
    - Status (aktiv, gelöscht)

5.2 Tag & Notebook Module

Aufgaben:

- Verwaltung von Tags
- Organisation von Notizen in Notizbüchern/Ordnern
- Mehrfachzuordnung von Tags zu Notizen

Schnittstellen:

- Nutzung durch Note Management Modul
- Abfragen durch Search Modul

Datenmodelle:

- Tag
- Notebook
- Zuordnungstabellen zwischen Notizen und Tags

5.3 Search Modul

Aufgaben:

- Lokale Volltextsuche
- Filterung nach Tags, Ordnern und Status
- Schnelle Ergebnisbereitstellung (Offline-First)

Schnittstellen:

- Lesezugriff auf Persistenzschicht
- Aufruf durch UI-Schicht

5.4 Revision & History Module

Aufgaben:

- Versionierung von Notizen
- Wiederherstellung früherer Zustände
- Nachvollziehbarkeit von Änderungen

Schnittstellen:

- Integration in Note Management Modul
- Zugriff auf Persistenzschicht

5.5 Trash Module

Aufgaben:

- Zwischenspeicherung gelöschter Notizen
- Wiederherstellung oder endgültige Löschung

Schnittstellen:

- Zusammenarbeit mit Note Management Modul
- Zugriff durch UI-Schicht

5.6 Persistence Module

Aufgaben:

- Lokale Speicherung aller Anwendungsdaten
- Sicherstellung von Konsistenz und Integrität
- Bereitstellung von Backup- und Restore-Funktionen

Technik:

- Lokale Datenbank / Dateisystem (projektspezifisch)

5.7 Sync Modul

Aufgaben:

- Synchronisation lokaler Änderungen mit einem externen Dienst
- Konflikterkennung und -behandlung
- Offline-Queue für Änderungen

6. Technische Spezifikationen

- **Programmiersprachen:** Java, Kotlin
- **Plattform:** JVM
- **Architekturprinzipien:** Modularität, Trennung der Schichten
- **Frameworks & Tools:** projektspezifisch definiert

Zentrale Algorithmen betreffen insbesondere:

- Such- und Filtermechanismen
- Revisionsverwaltung
- Konfliktbehandlung bei Synchronisation

7. Qualitäts- und Sicherheitsaspekte

Aspekte wie Performance, Sicherheit und Teststrategien werden in separaten Dokumenten behandelt.
Dieses SDD verweist auf die entsprechenden Konzepte und stellt deren technische Anknüpfungspunkte dar

.

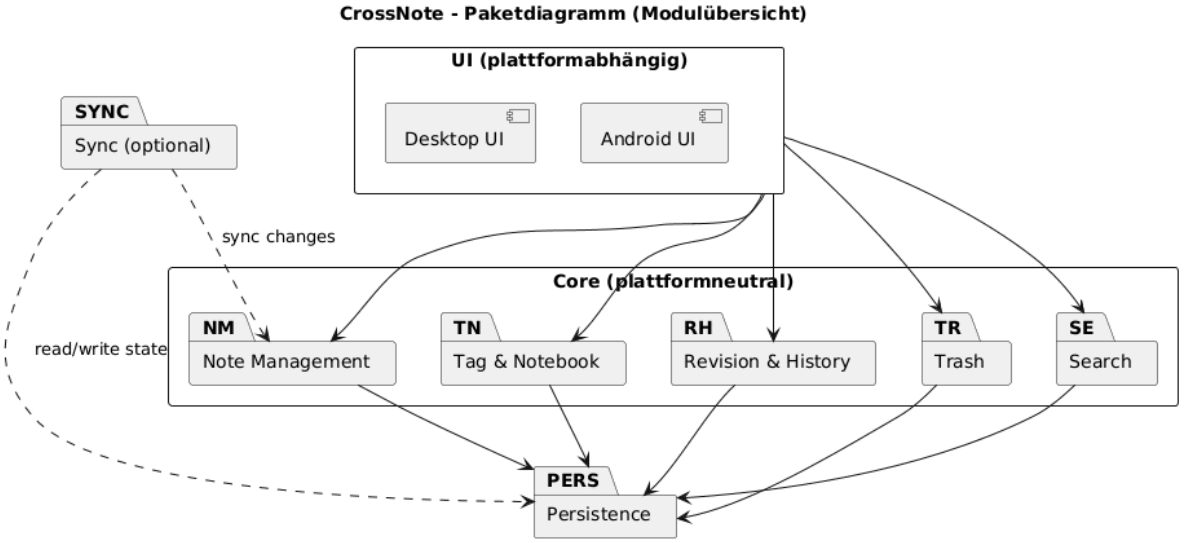
8. Zusammenfassung

Das Software-Design Document bildet die zentrale technische Grundlage für die Umsetzung von CrossNote.
Durch die klare Strukturierung der Module und Schnittstellen wird eine konsistente Implementierung, eine einfache Wartung sowie eine zukünftige Erweiterbarkeit der Anwendung sichergestellt.

Diagramme:

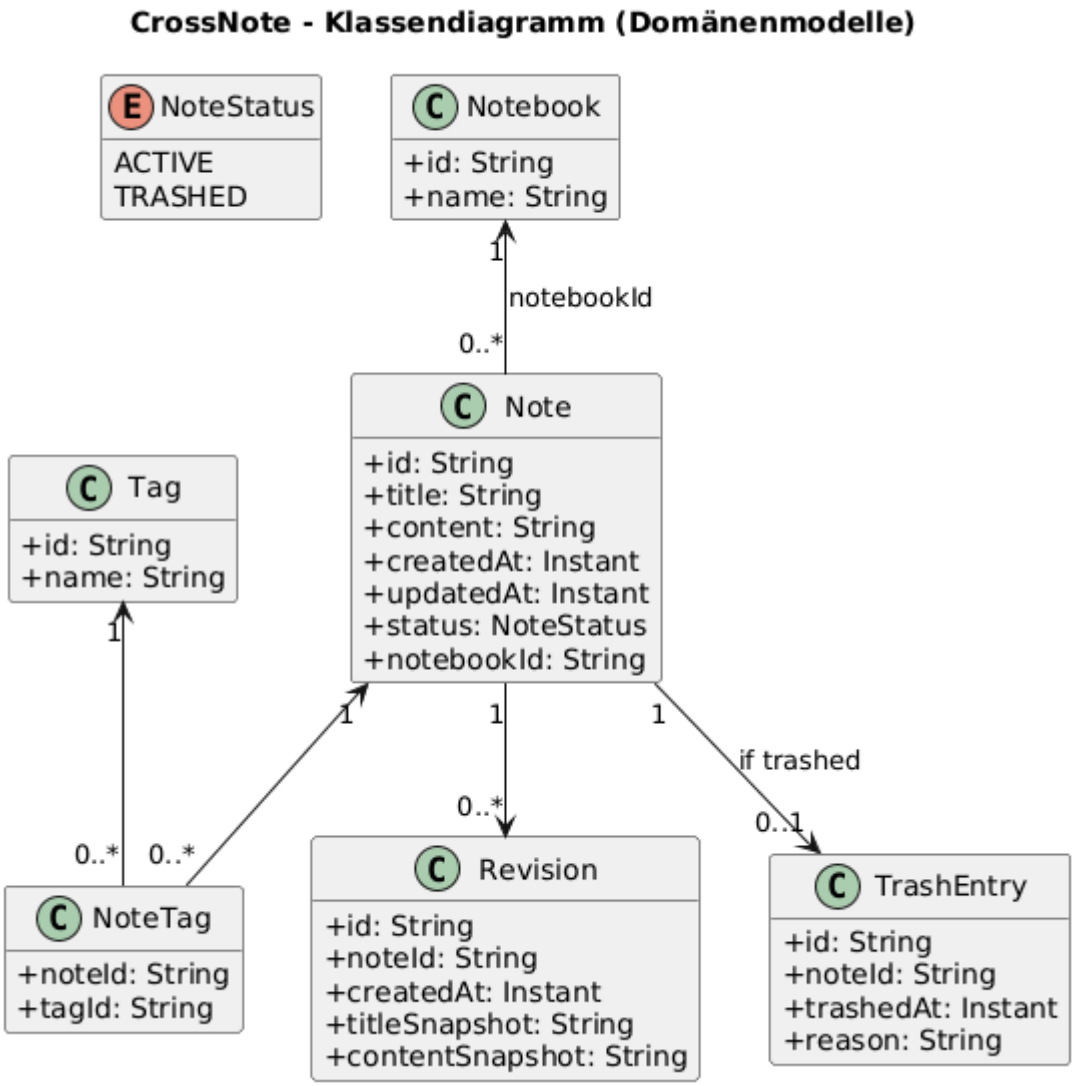
1) Paketdiagramm – Modulübersicht

Beschreibung:
Allgemeines Paketdiagramm, welches die verschiedenen Module und deren jeweiligen Implementierung Ort darstellen



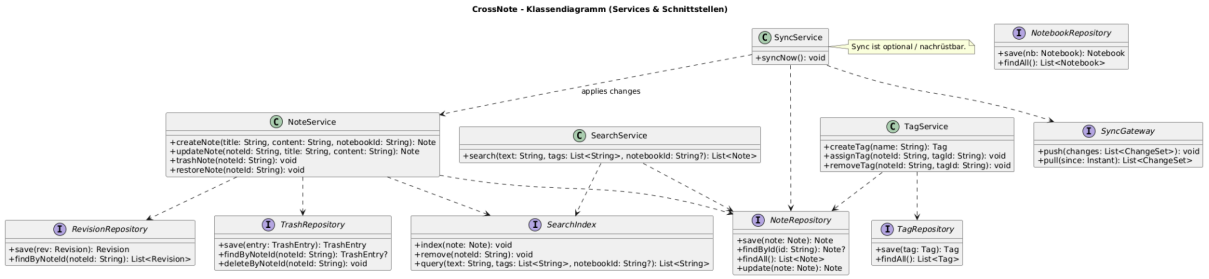
2) Klassendiagramm – Domänenmodelle (Datenmodelle)

Beschreibung:
Klassendiagramm für eine Notiz



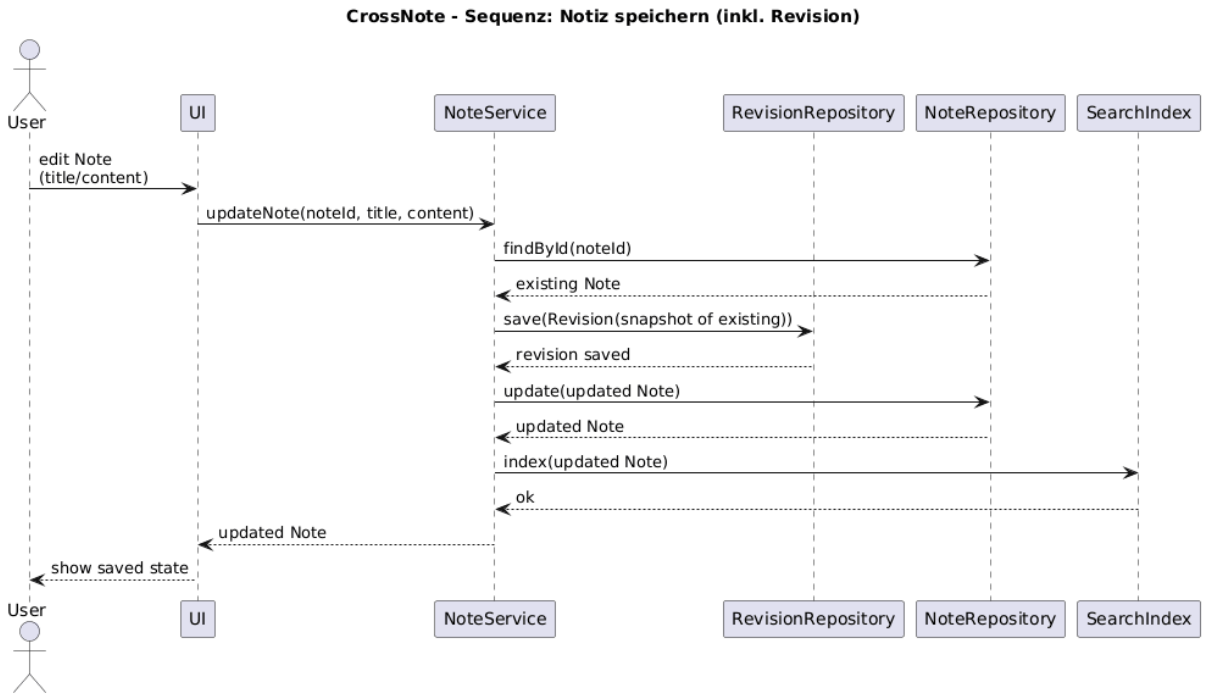
3) Klassendiagramm – Services, Use-Cases & Schnittstellen

Beschreibung:
Das Klassendiagramm zeigt die Aufteilung von CrossNote in Service-Klassen, die die Geschäftslogik der Anwendungsfälle umsetzen, sowie Interfaces, welche die technischen Schnittstellen zu Persistenz, Suche und optionaler Synchronisation definieren.
Die Services greifen ausschließlich über diese Interfaces auf externe Komponenten zu, wodurch eine klare Trennung von Logik und Implementierung sowie eine hohe Austauschbarkeit und Erweiterbarkeit erreicht werden.



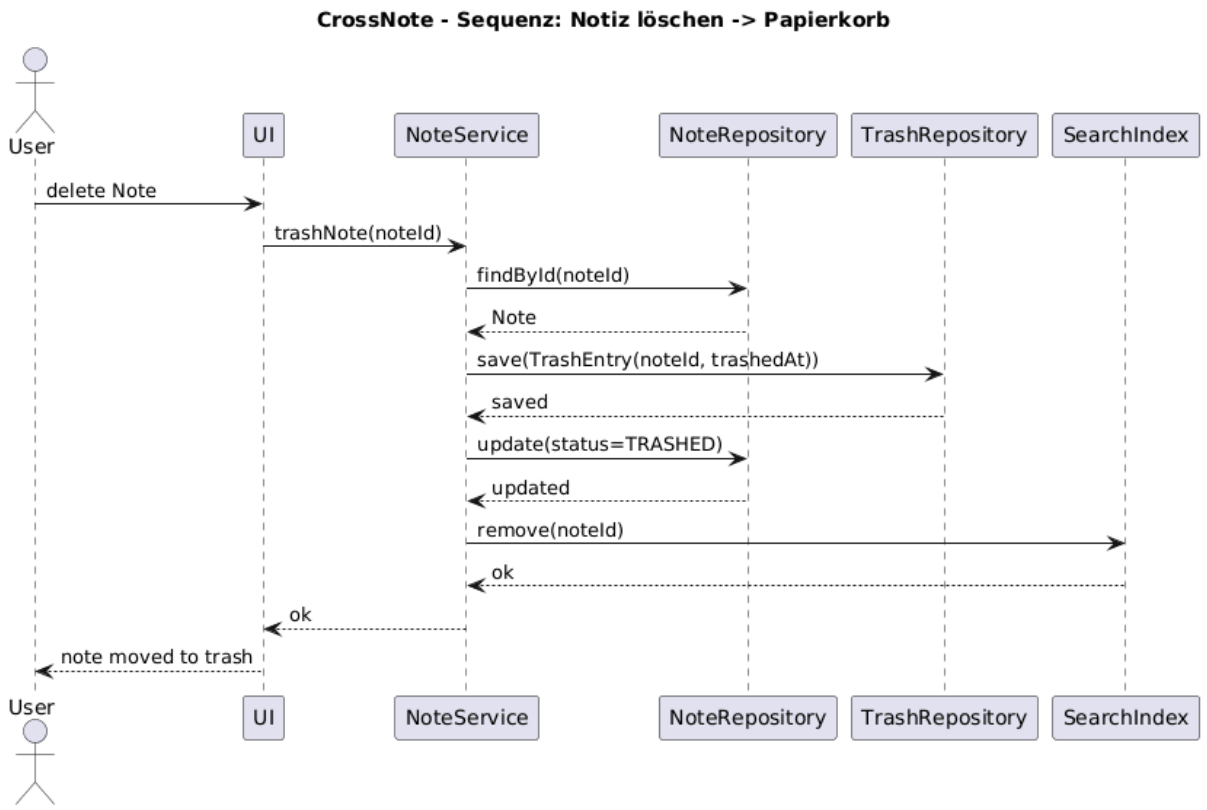
4) Sequenzdiagramm – Notiz speichern (inkl. Revision)

Beschreibung:
Funktionsablauf, welcher durchgegangen wird, wenn eine Notiz gespeichert wird



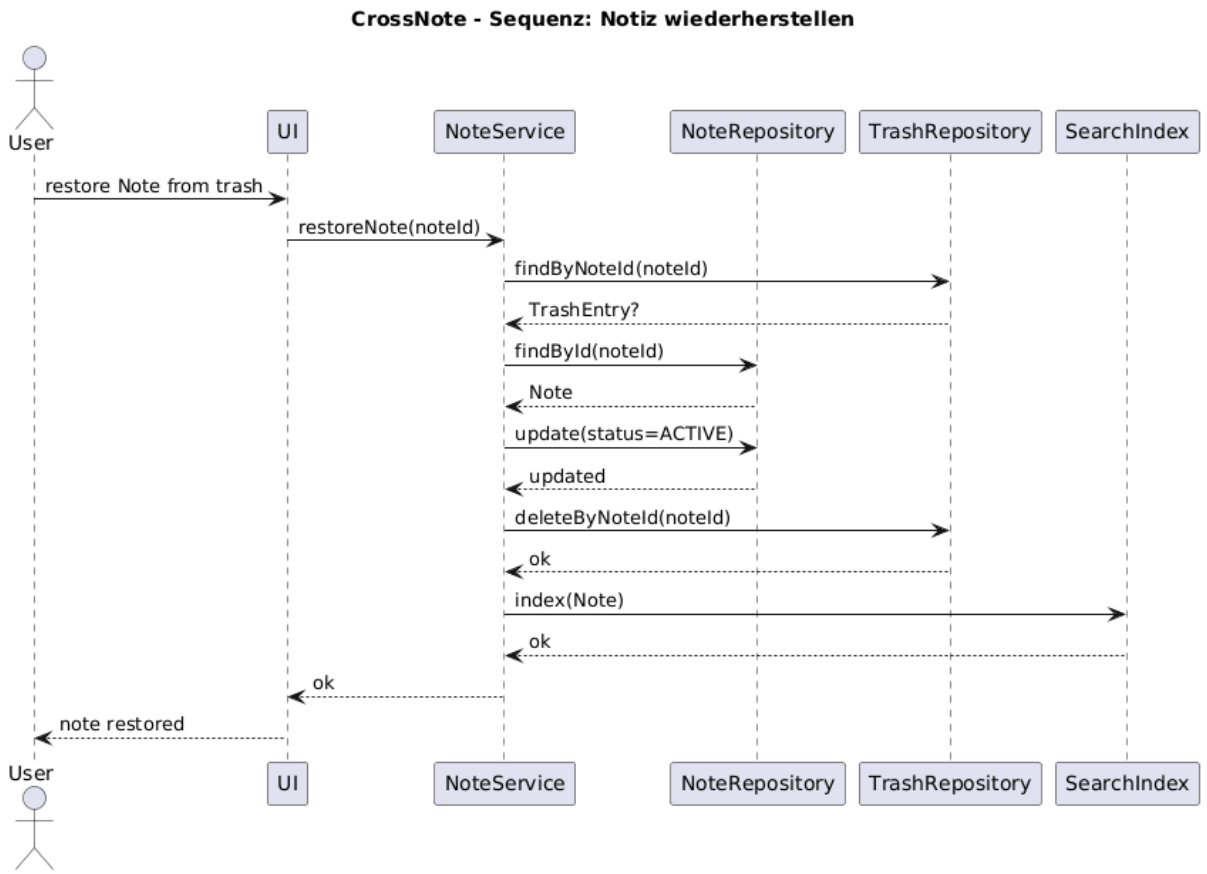
5) Sequenzdiagramm – Notiz löschen → Papierkorb

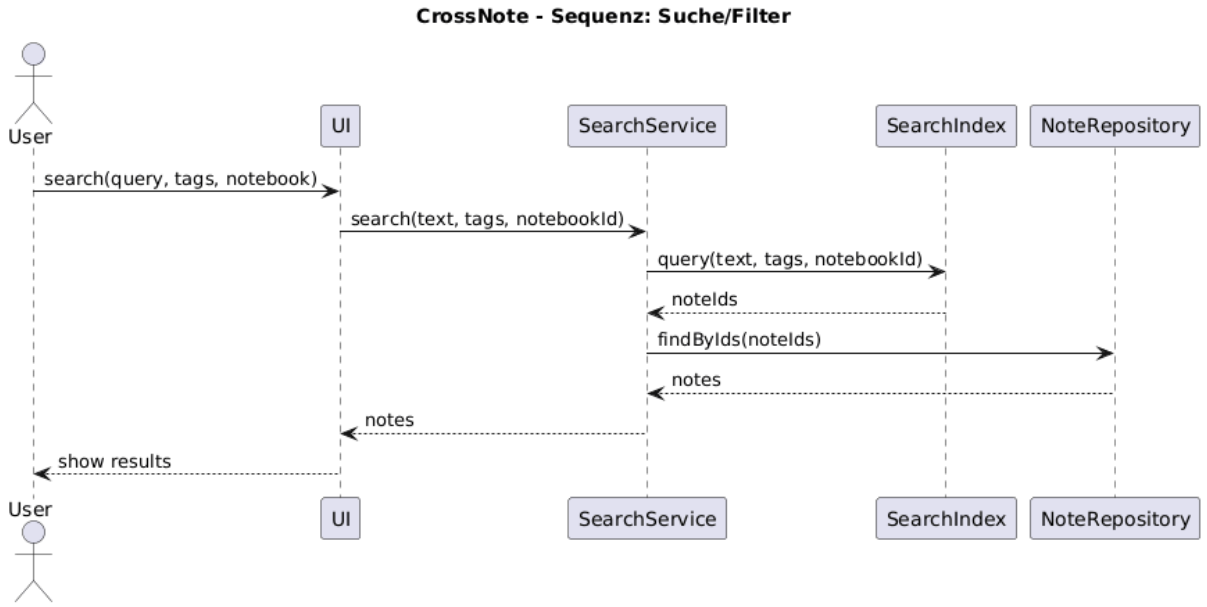
Beschreibung:
Funktionsablauf, welcher durchgegangen wird, wenn eine Notiz gelöscht wird



6) Sequenzdiagramm – Notiz wiederherstellen

Beschreibung:
Funktionsablauf, welcher durchgegangen wird, wenn eine Notiz wiederhergestellt wird



7) Sequenzdiagramm – Suche/Filter (Text, Tags, Notebook)

Beschreibung:
Funktionsablauf, welcher durchgegangen wird, wenn nach einer Notiz gesucht wird (mit allen Möglichkeiten)